DEVELOPERS DAY '26

Develop Ideas into Reality

Dev & Design Presents

HACKATHON

Build. Break. Innovate.

WEB DEVELOPMENT HACKATHON — PROBLEM STATEMENT

LIGMA - Let's Integrate Groups, Manage Anything
Virtual Collaboration Workspace

Note to Participants

You are completely free to use any AI tool for support. Although, the evaluation (excluding frontend) will be based on your design knowledge and the architectural design decisions. **USE OF PAID 3RD PARTY INTEGRATIONS WILL RESULT IN IMMEDIATE DISQUALIFICATION.**

---

01 Background

Modern remote teams operate across fragmented toolchains — a whiteboard for ideation, a task manager for execution, a chat tool for decisions. The cognitive cost of context-switching during a live brainstorm kills momentum: a great idea scribbled on a sticky note becomes a lost action item because nobody transferred it to the project tracker.

Existing tools either solve the canvas problem (Figma, Miro) or the task management problem (Notion, Linear) — but none bridge the gap in real time, at the moment the idea is born.

---

02 The Challenge

Build **LIGMA** — a real-time collaborative workspace that bridges ideation and execution. Teams brainstorm on a shared infinite canvas; the platform automatically extracts intent from canvas content and populates a live task board — no copy-paste, no context switching.
LIGMA is intentionally scoped: it is not a full Figma clone. It is a purpose-built brainstorming tool that produces structured, actionable output the moment a session ends.

---

03 Core Requirements

The Canvas

- Infinite canvas with sticky notes, freehand drawing, shapes, and text blocks.
- Multiple users can edit simultaneously in real time (no page refresh).
- Each canvas element (node) is an independently addressable object.
- Cursor presence — each user's cursor is visible to all participants.

Intent-Aware Task Extraction

- As users write on canvas nodes, an AI layer classifies intent: action item, decision, open question, or reference.
- Nodes tagged as action items automatically appear in a structured Task Board panel.
- Tasks carry the original author, timestamp, and a link back to the canvas node — no data is duplicated.

Node-Level Access Control

- Individual canvas nodes can be locked to specific roles (e.g., Lead, Contributor, Viewer).
- A "Lead" can lock the architecture diagram while contributors can still comment on it.

Append-Only Event Log

- Every mutation to the canvas is stored as an immutable event. The logs can be viewed in a bar on the side.

Real-Time WebSocket Management

- The system must handle multiple concurrent WebSocket connections, broadcast canvas deltas (not full state) efficiently, and recover gracefully when a client reconnects after a drop — replaying only missed events.

Deployment of the module on Render

- In order to be judged it's important to deploy the finished project onto Render.

---

04 Technical Challenges

The following challenges go beyond standard CRUD. Teams must demonstrate understanding of the underlying problem, not just a working UI.

1. **Challenge 01: Conflict Resolution (CRDT / OT)**
   When two users simultaneously edit the same canvas node, implement proper merge logic — not "last write wins." The canvas must converge to a consistent, correct state across all connected clients. This is what Figma and Google Docs actually do under the hood.
2. **Challenge 02: Node-Level RBAC**
   Most apps enforce permissions at the room level. LIGMA requires per-node permissions. Individual canvas elements must carry their own ACL, enforced both client-side (UI affordances) and server-side (mutation validation).
3. **Challenge 03: Intent-Aware Task Extraction**
   As users write on canvas nodes, an AI layer classifies intent: action item, decision, open question, or reference.
4. **Challenge 04: Append-Only Event Log**
   Append-Only Event Log Every mutation to the canvas is stored as an immutable event. The logs can be viewed in a bar on the side.
5. **Challenge 05: Real-Time WebSocket Management**
   Real-Time WebSocket Management The system must handle multiple concurrent WebSocket connections, broadcast canvas deltas (not full state) efficiently, and recover gracefully when a client reconnects after a drop — replaying only missed events.
6. **Challenge 06: Deployment of the module on Render**
   Deployment of the module on Render In order to be judged it's important to deploy the finished project onto Render.

---

05 Creative Features (Bonus)

These features are optional but reward teams that plan their architecture well in Stage 1:

- **Presence Heatmap:** Visual overlay showing which canvas zones received the most edits and attention during the session.
- **Time-Travel Replay:** Scrub through session history like a video timeline — watch the brainstorm unfold step by step.
- **AI Summary Export:** One-click export of the canvas content into a structured brief: decisions made, tasks assigned, open questions.
- **Presence Zones:** Named focus areas on the canvas that display which team members are currently active in each zone.

Note to participants: You are not expected to build all four creative features. One well-implemented bonus feature with a clear architecture is worth more than four half-finished ones.

06 Evaluation Rubric

Point Distribution Overview

Category	Real-Time Collab	Core Features	Architecture	UI / UX	Innovation	TOTAL
Max Pts	25	25	20	15	15	100

Detailed Scoring Criteria

Criterion	Pts	☑ Full Marks	✕ Zero Marks
CATEGORY 1 Real-Time Collaboration & Conflict Resolution // 25 pts			
Multi-user canvas sync	10	Open two browser tabs with different user sessions. Draw a shape in Tab A — it appears in Tab B within a reasonable period.	Changes in Tab A are only visible in Tab B after a manual reload, or never appear at all.
Conflict resolution	10	Both users type in the same text node simultaneously. Both tabs show identical merged text. README explains strategy.	One user's text silently overwrites the other's, or README has no explanation of conflict handling.
Cursor presence	5	Each connected user's cursor is visible on all other screens, labelled, updating smoothly.	No cursor presence, or cursors delay more than 2 seconds.

CATEGORY 2 Core Feature Implementation // 25 pts			
AI intent extraction	10	Type an action item. Within 3 seconds, Task Board shows new task with author and link automatically.	Task creation requires manual click, no task appears, or Task Board panel is absent.
Task Board integration	8	Task Board is live for all users. Clicking a task scrolls canvas to the originating node.	Task Board requires reload to update, or tasks have no link back to the canvas node.
Node-level RBAC	7	Viewer edit is blocked in UI and on server (WebSocket error). Live role demotion takes effect without reload.	RBAC enforced only in UI (WebSocket bypass possible), or permissions require reload.
CATEGORY 3 Technical Architecture & Code Quality // 20 pts			
Event-sourced arch	8	Mutations stored as immutable events with timestamp/ID. Deleting a node inserts an event; doesn't remove history.	Mutations overwrite state in place. No event log. State cannot be reconstructed from history.
API design & quality	7	REST/GraphQL returns structured JSON. Codebase has clear separation of concerns (routes, services, data).	Logic in a single file, inconsistent structures, or endpoints fail outside the browser.
README & docs	5	Contains architecture diagram/description and explanation of technical choices (CRDT, Event-sourcing).	No README, or it is a default scaffold template with no project-specific content.

CATEGORY 4 UI / UX // 15 pts			
Canvas usability	8	User can add/move sticky notes, connect nodes, and assign tags without explanation.	Core interactions require explanation; controls are hidden behind unlabelled icons.
Responsiveness	4	Canvas and Task Board are functional/readable. No horizontal scrollbar on the main layout.	Layout breaks, overlaps, or key controls are cut off and unreachable at standard widths.
Visual consistency	3	Consistent palette, font hierarchy, and spacing.	Conflicting font sizes, mismatched button styles, or unstyled default browser elements.
CATEGORY 5 Innovation & Creativity // 15 pts			
Bonus feature	8	One creative feature is fully functional and demoed live (not just a UI mockup).	Feature shown only as static screenshot, a disabled button, or crashes during demo.
Approach uniqueness	7	Team articulates non-obvious architectural decisions with clear reasoning during Stage 1.	Team cannot explain technical choices; codebase is a scaffold with no unique design decisions.

07 Judging Notes

- **Partial credit** is independent per criterion. If conflict resolution works but the README has no explanation, judges award the WebSocket points but dock the evolution rationale points separately.
- **Server-side enforcement** is mandatory for security criteria. Client-only guards earn zero on RBAC — judges will test by sending a raw WebSocket or curl request.
- **Live demo is required.** Screenshots and videos do not substitute for a working demo on any criterion marked with a curl / DevTools / tab-switching test action.
- **Stage 1 (Architecture Presentation)** scores feed into Category 5. Teams that plan well and articulate their choices clearly earn the uniqueness points even if their implementation is incomplete.